

Hardware Acceleration of Canny Edge Detection on the Kria KV260v

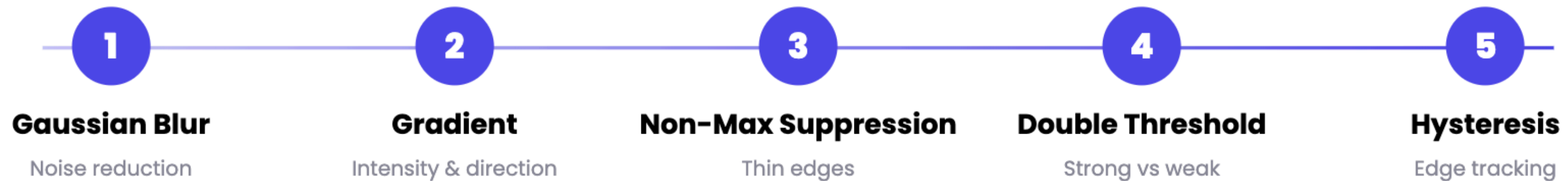
Team members: Luke McCallum (z5591695) · Kaashvi Goel (z5623123) · Rahul vignesh(z5611329) · Thomas Saw (z5587733)

Repository: github.com/superkooks/comp4601

3 July 2026

Problem Outline

Canny edge detection is a **5-stage, multi-pass, per-pixel pipeline**: two 2D convolutions (Gaussian, Sobel) plus three neighbourhood operations per frame.



WHY IT MATTERS?

- Canny is the most widely used edge detector — foundational to autonomous driving, medical imaging, and industrial inspection.
- Computationally intensive: five sequential stages applied to every pixel of every frame

THE BOTTLENECK

- CPU implementations cannot sustain real-time performance at high resolutions — parallelism is limited.
- FPGAs exploit massive pixel-level parallelism and deep pipelining — prior work reports up to **100× speedup**.

The problem: how to efficiently map the full Canny pipeline onto FPGA hardware using High-Level Synthesis to achieve real-time edge detection.

Project Specific Aims

From OpenCV baseline to a real-time HLS pipeline on the Kria KV260

1

Software baseline

Measure Canny latency and FPS using OpenCV on the Kria KV260 ARM processor (PS).

2

HLS implementation

Implement all five Canny stages as synthesisable HLS C/C++ kernels in Vitis HLS.

3

Correctness verification

Verify through C-simulation against the OpenCV reference output.

4

Hardware deployment

Deploy on KV260 programmable logic (PL) with live webcam input and real-time output display.

5

Benchmark & optimise

Compare against cv2.Canny() on PS ARM cores. Metrics: fps, per-frame latency, and PL resource utilisation (LUT / FF / BRAM / DSP)

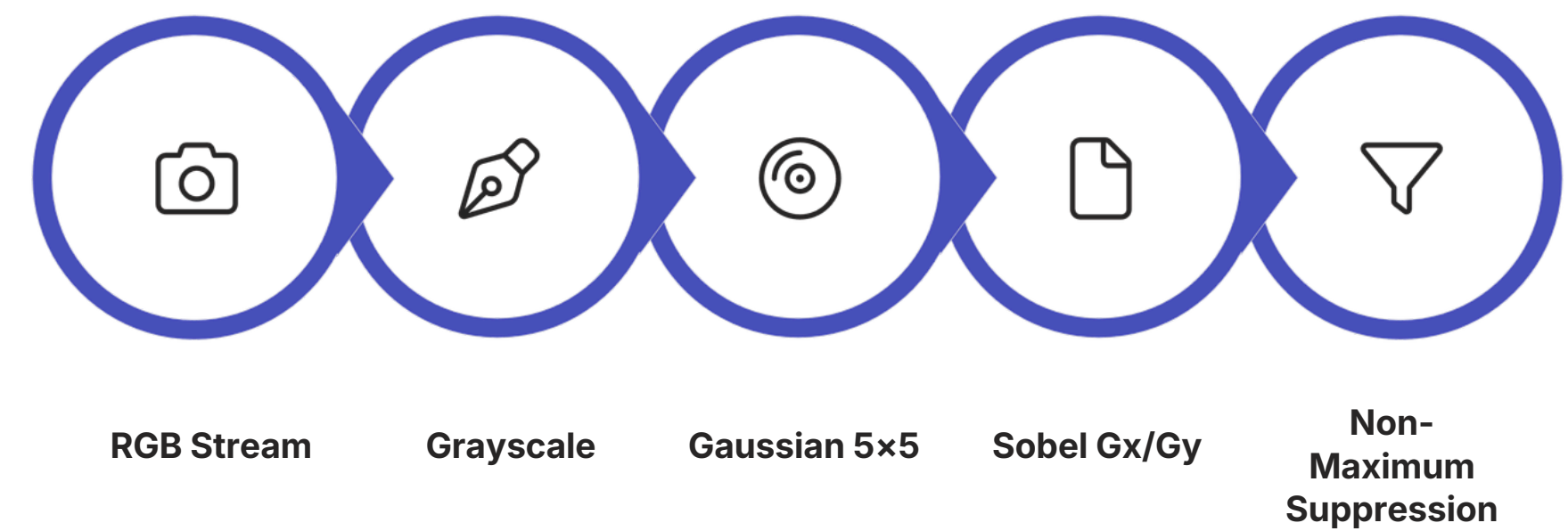
PERFORMANCE TARGET

≥30 FPS

Real-time or better-than-real-time edge detection on standard-resolution video.

Algorithm to be Accelerated:

Canny Edge Detection



Early Stages

- **Grayscale:** reduce 24-bit RGB to 8-bit luminance (shift-add weights, no floating point)
- **Gaussian blur:** 5x5 convolution suppresses noise before differentiation (Canny is highly noise-sensitive)
- **Sobel gradients:** magnitude approximated $|G| = |G_x| + |G_y|$ to avoid sqrt in hardware; direction $\text{atan2}(G_y, G_x)$ quantised to 4 bins (0, 45, 90, 135°) so comparison logic replaces trigonometry

Later Stages

- **Non-maximum suppression:** keep a pixel only if its magnitude exceeds both neighbours along the quantised gradient direction — thins edges to 1 px
- **Double threshold:** classify pixels as strong ($\geq \text{hthr}$), weak ($\geq \text{lthr}$), or suppressed
- **Hysteresis:** promote weak pixels to edges only if 8-connected to a strong pixel

Problems with software	FPGA solution
Software implementations process each pixel sequentially	FPGAs allow for hardware acceleration techniques such as pipelining that could significantly improve performance
Stages like Gaussian filtering and gradient calculation contain convolution operations, which would in practice be many arithmetic operations for each pixel	An FPGA implementation would make use of dedicated hardware to perform multiplications and additions in parallel, potentially achieving up to 1 pixel per clock cycle
Memory would need to be repeatedly accessed as each pixel is read, processed and then written back to memory	Each pixel can flow directly through the hardware between different stages, completely avoiding unnecessary memory accesses

Sobel Kernels

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

Every stage is a local (at most 5x5) neighbourhood operation — ideal for line-buffer streaming; no full-frame storage needed on-chip.

Investigation So Far

Our investigation covered four areas. First, we analysed the Canny algorithm itself, stage by stage, to understand which parts map naturally onto hardware and which parts resist it. Second, we studied the streaming design techniques that HLS image pipelines rely on, since an FPGA cannot simply hold a frame in memory and loop over it the way software does. Third, we examined the KV260 platform to understand how work should be divided between its ARM processors and its FPGA fabric. Finally, we reviewed existing open-source HLS implementations of Canny as background reading, to sanity-check our architectural thinking against designs that have actually been built

Algorithm Interface:

```
void canny_edge_detection(hls::stream<rgb_pixel_t> &axis_in,  
                        hls::stream<pixel_t> &axis_out,  
                        uint8_t hthr, uint8_t lthr);
```

➤ *We will stream in pixel values (synthesizes as AXIS), aggregate them into line buffers and pipeline each line of the image. The pixels will also be streamed out.*

Targets & Risks Identified after investigation :

Throughput Target

Targeting one pixel per clock:
~150 Mpix/s at a 150 MHz PL
clock vs the 27.6 Mpix/s needed
for 720p30. To be confirmed by
synthesis reports in M3.

Hardest Stage

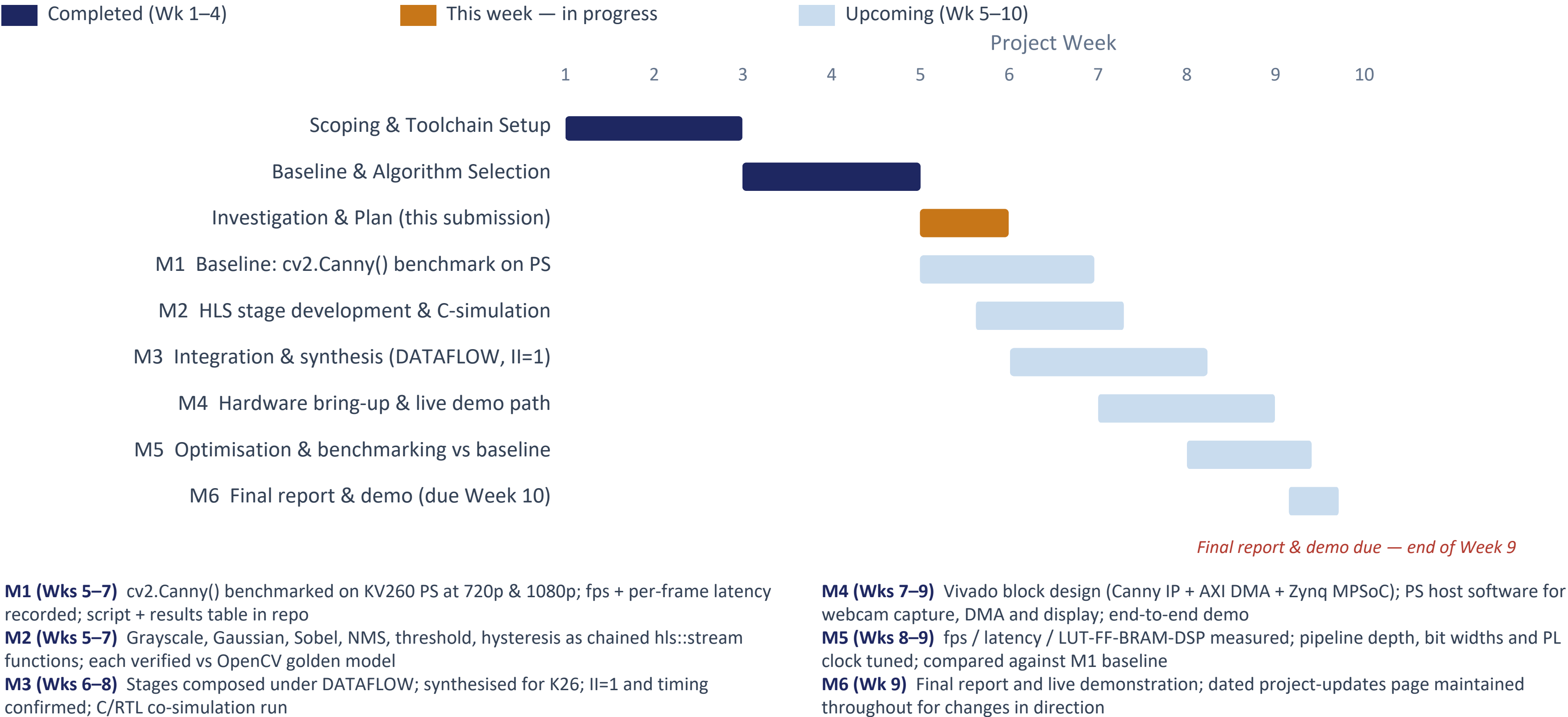
Hysteresis is the only non-local
stage - edge connectivity can
propagate across the image. We'll
prototype it first in C simulation,
with a single-pass windowed
version as fallback.

Key Findings :

- **Algorithm structure:** the algorithm progresses in stages, with each stage processing a single framebuffer at a time
- **Target performance:** close to 1 pixel/clock via pipeline processing across chained stage functions.
- **Convolution pattern:** line buffers + sliding window so 3×3/5×5 neighbourhoods are formed from a raster pixel stream without storing full frames in BRAM
- **Integration with host:** should be able to integrate our kernel with OpenCV in host software for webcam demo

Project Plan — Milestones & Timeline

Week 5 of 11 — Weeks 1–4 complete · Investigation & Plan due now · M1–M6 span Weeks 5–10



Risks & Contingencies

R1 — Timing/Resource Failure on K26

The resource requirements to meet the desired clock are unknown.

Contingency: reduce resolution to 720p, lower PL clock.

R2 — Integration with Host Application

May not be possible to integrate with OpenCV for demo.

Contingency: use static image/video file instead of webcam for demo

R3 — Hysteresis Streaming Difficulty

Hysteresis stage is not purely local (edge connectivity can propagate) — hardest stage to stream.

Contingency: multi-pass windowed hysteresis approximation, accepting minor quality loss versus OpenCV.

R4 — Team Availability Near Deadline

Due to obligations to other subjects, not all team members may be available up to the deadline.

Contingency: milestone checkpoints at each week boundary, may need to reassign work.

Roles of Team Members

Luke McCallum — z5591695

- Host application / OpenCV capture
- Double threshold, hysteresis stages (+ C simulation)

Rahul Vignesh — z5611329

- Gaussian stage (+ C simulation)
- OpenCV golden-model verification

Thomas Saw — z5587733

- HLS pipeline architecture (DATAFLOW top level)
- Grayscale, Sobel stages (+ C simulation)

Kaashvi Goel — z5623123

- Non-maximum suppression stage (+ C simulation)
- Baseline and hardware benchmarking (fps, latency, resource utilisation)



All members: weekly milestone reviews; shared responsibility for the final demo and report.

Team point of contact: Luke McCallum <z5591695@ad.unsw.edu.au>